Q.NO :9

**Question description**

Rathik organized technical round interview in Macrosoft for the set of computer science candidates.

The problem is to perform Implement a stack using single queue. you have to use queue data structure, the task is to implement stack using only given queue data structure.

Rathik have given the deadline of only 5 minutes to complete the problem.

Can you Help the candidates to complete the problem within the specified time limit ?

**Function Description**

x is the element to be pushed and s is stack

**push(s, x)**

1) Let size of q be s.

2) Enqueue x to q

3) One by one Dequeue s items from queue and enqueue them.

Removes an item from stack

**pop(s)**

1) Dequeue an item from q

Constraints

0 < n, m < N

1 < arr[i] < 1000

**Input Format:**

First line indicates n & m, where n is the number of elements to be pushed into stack and m is the number of pop operation need to be performed

next line indicates the n number stack elements

**Output Format:**

First line indicates top of the element of the stack

second line indicates the top of the element after the pop operation

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 2
1 2 3 4 5
```

EXPECTED OUTPUT

```
top of element 5
top of element 3
```

Test Case 2

INPUT (STDIN)

```
15 4
9 8 7 6 5 5 6 7 8 3 11 2 3 4 5
```

EXPECTED OUTPUT

```
top of element 5
top of element 11
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void Stack::push(int val)
```

Test Case 2

KEYWORD

```
q.push(val)
```

Test Case 3

KEYWORD

```
void Stack::pop()
```

Test Case 4

KEYWORD

```
q.pop();
```

Code:

```
#include <stdio.h>

#define MAX 1000
```

```c
int queue[MAX];
int front = 0;
int rear = 0;
int size = 0;



void enqueue(int x)
{
queue[rear] = x;
rear++;
size++;
}



int dequeue()
{
int x = queue[front];
front++;
size--;
return x;
}

void push(int x)
{
int oldSize = size;

enqueue(x);
for (int i = 0; i < oldSize; i++)
{
int temp = dequeue();
enqueue(temp);
}
}int pop()
{
return dequeue();
}

int main()
{
int n, m;

scanf("%d %d", &n, &m);

int x;
for (int i = 0; i < n; i++)
{
scanf("%d", &x);
push(x);
}
```

```
printf("top of element %d\n", queue[front]);



for (int i = 0; i < m; i++)
{
pop();
}



printf("top of element %d\n", queue[front]);


return 0;
}
```

Output:

Test _case 1:

```
5 2
1 2 3 4 5
top of element 5
top of element 3
```

Test _case 2:

```
15 4
9 8  7  6 5 5 6 7 8 3 11 2 3 4 5
top of element 5
top of element 11
```

Q.no 10

| DS | Session | Stack | Question Information | Level 1 | Challenge 50 |
|---|---|---|---|---|---|

**Question description**

Hassan gets a job in a software company in Hyderabad. The training period for the first three months is 20000 salary. Then incremented to 25000 salaries.

Training is great but they will give you a programming task every day in three months. Hassan must finish it in the allotted time. His teammate Jocelyn gives him a task to complete the concept of Postfix to Prefix Conversion for a given expression. can you help him?

**Functional Description:**

- Read the Prefix expression in reverse order (from right to left)
- If the symbol is an operand, then push it onto the Stack
- If the symbol is an operator, then pop two operands from the Stack
  Create a string by concatenating the two operands and the operator after them.
  **string = operand1 + operand2 + operator**
  And push the resultant string back to Stack
- Repeat the above steps until end of Prefix expression.

**Constraints**

the input should be a expressions

**Input Format**

Single line represents the postfixed expressions

**Output Format**

Single line represents the prefix expression

Logical Test Cases

Test Case 1

INPUT (STDIN)

ABC/-AK/L-*

EXPECTED OUTPUT

*-A/BC-/AKL

Test Case 2

INPUT (STDIN)

AB+CD-*

EXPECTED OUTPUT

*+AB-CD

Mandatory Test Cases

Test Case 1

KEYWORD

string postToPre(string post_exp)

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

5

Test Case 2

TOKEN COUNT

220

Test Case 3

NLOC

46

Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>


char stack[100][100];
int top = -1;
```